ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

INTRODUCTION TO TECHNICAL PROGRAMMING

PAPER 1 · PRACTICE C · MARKING GUIDELINE

MARKS: 120

Marking guideline for the Academic Central NC(V) Level 3 Introduction to Technical Programming paper 1 · practice c.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Code is not marked strictly on syntax: give marks for correct logic and structure. Accept other correct answers, other variable names and other correct flowchart layouts.

SECTION A

QUESTION 1

1.1 A ✓
1.2 B ✓
1.3 C ✓
1.4 D ✓
1.5 A ✓
1.6 B ✓
1.7 C ✓
1.8 B ✓
1.9 A ✓
1.10 D ✓

(10)

[10]

QUESTION 2

2.1 True ✓
2.2 False ✓
2.3 True ✓
2.4 False ✓
2.5 True ✓

(5)

[5]

QUESTION 3

3.1 E ✓
3.2 C ✓
3.3 A ✓
3.4 G ✓
3.5 D ✓
3.6 I ✓
3.7 B ✓
3.8 J ✓
3.9 F ✓
3.10 H ✓

(10)

[10]

QUESTION 4

4.1 Terminator (start / end) ✓

4.2 Input / output ✓

4.3 Process ✓

4.4 Decision ✓

4.5 Flow line (arrow) ✓

4.6 Connector ✓

(6)

[6]

TOTAL SECTION A: [31]

SECTION B

QUESTION 5

5.1.1 A mistake that breaks the grammar rules of the programming language ✓, so the program cannot be compiled or run ✓ (e.g. a missing semicolon). (2)

5.1.2 A named memory location ✓ that stores a value which can change while the program runs ✓. Example ✓✓: int score = 0; (4)

5.1.3 The CPU (central processing unit / processor) ✓✓ (2)

5.1.4 Hard disk drive / solid-state drive (SSD) / flash drive ✓✓ (2)

5.1.5 A finite set of clear, step-by-step instructions ✓ that solves a problem or completes a task ✓. (2)

5.2 Focusing on the important information and ignoring details that are not needed ✓, so the problem is simpler to solve ✓. (2)

5.3 High-level: close to human language ✓, easy to read and write, portable between computers, must be compiled or interpreted (e.g. C++, Python) ✓. Low-level: close to machine code / assembly ✓, hard to read, specific to one processor but very fast and gives direct control of the hardware ✓. (4)

5.4 A local variable is declared inside a function or block ✓ and can only be used there ✓; a global variable is declared outside all functions ✓ and can be used anywhere in the program ✓. (4)

5.5 Any FOUR in order ✓✓✓✓: analyse / define the problem; design the solution (algorithm, pseudocode, flowchart); write the code; test and debug; document; maintain. (4)

[26]

QUESTION 6

Any ELEVEN relevant points ✓ (one mark each), e.g.: one student creates a repository on GitHub for the project ✓ and adds the others as collaborators ✓; each student clones the repository to their laptop ✓; each student works on their own branch (scores, ranking, screens) ✓ so they do not overwrite each other's work ✓; they commit small changes often ✓ with clear commit messages ✓; they push their branches to GitHub ✓ and pull regularly to get the others' latest work ✓; finished work is merged through a pull request ✓ that a team member reviews ✓; Git shows merge conflicts when two people change the same lines, which they resolve together ✓; the history lets them see who changed what and go back to an older version if something breaks ✓; GitHub Issues can be used to list and share tasks and bugs ✓; GitHub also keeps a backup of the code in the cloud ✓. (11)

[11]

QUESTION 7

7.1 secret – the random number to be guessed ✓; tries – how many guesses have been made ✓; guess – the number the user entered ✓ (3)

7.2 The block "pick random 1 to 20" ✓ gives a random whole number from 1 to 20, which is stored in secret ✓. (2)

7.3 "repeat until" ✓; it stops when guess is equal to secret (the user guessed correctly) ✓. (2)

7.4 It adds 1 to the variable tries ✓ each time the user makes a guess, so it counts the guesses ✓. (2)

7.5 Too high! ✓✓ (2)

7.6 Too low! ✓ Too high! ✓ Correct! Tries: 3 ✓ (3)

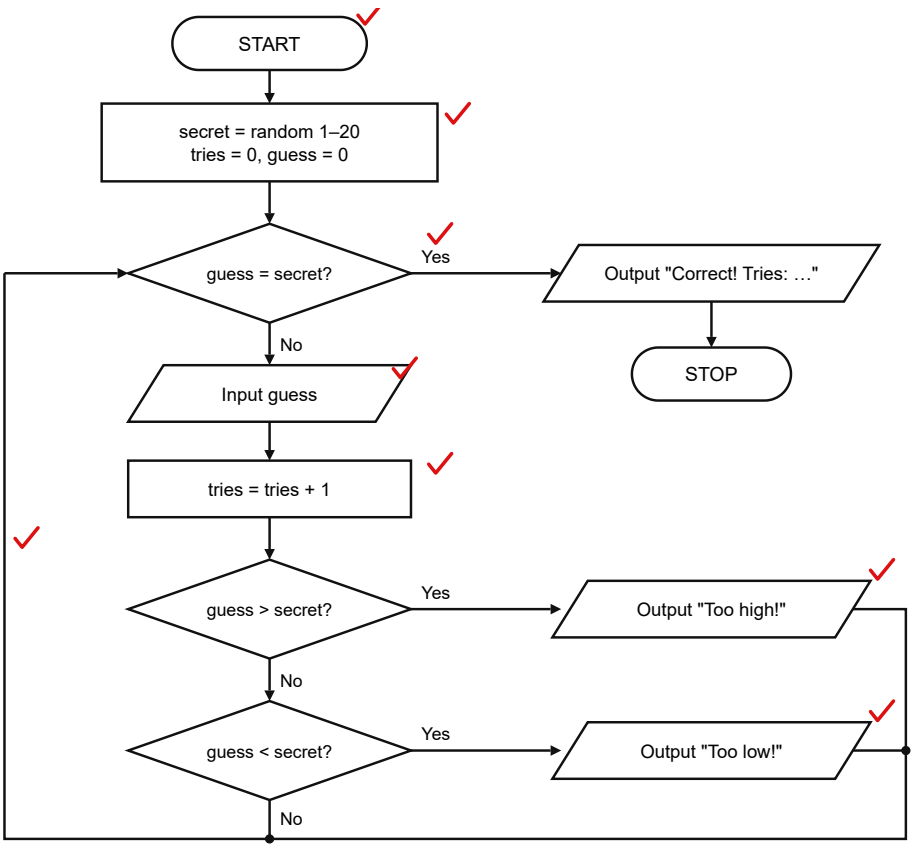
7.7 Libraries and main() ✓; random secret 1–20 (rand() % 20 + 1) ✓; variables declared ✓; do … while loop ✓ with condition guess != secret ✓; input of guess ✓; tries counted ✓; "too high / too low" messages and final message with tries ✓, e.g. (8)

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    srand(time(0));                    // different numbers each run
    int secret = rand() % 20 + 1;      // 1 to 20
    int guess = 0, tries = 0;
    do {
        cout << "Guess a number from 1 to 20: ";
        cin >> guess;
        tries++;
        if (guess > secret) cout << "Too high!" << endl;
        if (guess < secret) cout << "Too low!" << endl;
    } while (guess != secret);
    cout << "Correct! Tries: " << tries << endl;
    return 0;
}
```

7.8 "repeat 10" always runs exactly 10 times, while "repeat until" runs until a condition becomes true ✓. The number of guesses is not known in advance, so the loop must stop only when the guess is correct ✓. (2)

7.9



(Start 1; set secret (random), tries = 0, guess = 0 1; decision guess = secret? 1 (No → into the loop, Yes → final message); input guess 1; tries = tries + 1 1; decision guess > secret → "Too high!" 1; decision guess < secret → "Too low!" 1; loop back, then output "Correct! Tries: …" and stop 1)

(8)

[32]

QUESTION 8

8.1 A while loop ✓; −1 is a sentinel (stop / end) value ✓ (2)

8.2 The first cin gets a value so the condition can be tested before the loop starts ✓; the second gets the next value at the end of each repetition so the loop can test it again ✓. (2)

8.3 It adds 1 to the variable good ✓, counting one more good battery ✓. (2)

8.4 Weak ✓✓ (6,4 ≥ 6,4 but less than 7,0) (2)

8.5 good = 1, weak = 1, flat = 1 ✓✓ (7,4 good; 6,2 flat; 6,8 weak) (2)

8.6 $(7{,}4 + 6{,}2 + 6{,}8) \div 3 = 20{,}4 \div 3$ ✓ $= 6{,}8$ V ✓ (2)

8.7 If no batteries were entered, count is 0 and dividing by 0 would cause an error / a wrong result ✓✓. (2)

8.8 No batteries tested. ✓✓ (2)

8.9 double ✓ – voltages have decimal parts (e.g. 6,8) ✓ (2)

8.10 sum += volts; ✓✓ (2)

[20]

TOTAL SECTION B: [89]

TOTAL: 120

END OF MARKING GUIDELINE